

COVER STORY

이진원

— Lee Jinwon

Edition Four — Building across layers.

Full-stack Engineer · Mobile · Backend · Embedded Sim

ljin2091@naver.com

GitLab ↗ (<https://lab.ssafy.com/ljin2091>)

ROLE

Full-stack Engineer

FOCUS

End-to-end systems

BASED

Seoul, KR

AVAILABLE

2026.06 +

About

Flutter 앱부터 Spring/Django 백엔드, Isaac Lab 시뮬레이션과 라즈베리파이 디바이스 통합까지 — 한 프로젝트의 끝에서 끝까지 책임지는 풀스택 엔지니어를 지향합니다.

#풀스택 통합

#디바이스-서버-앱 연동

#RL 시뮬레이션

#인프라 자동화

ROLE Full-stack Engineer 지향

INTEREST End-to-end product delivery, Cross-layer integration, Reinforcement learning sims

MOTTO *“Build end-to-end. Bridge the gaps. Ship as one.”*

Tech Stack

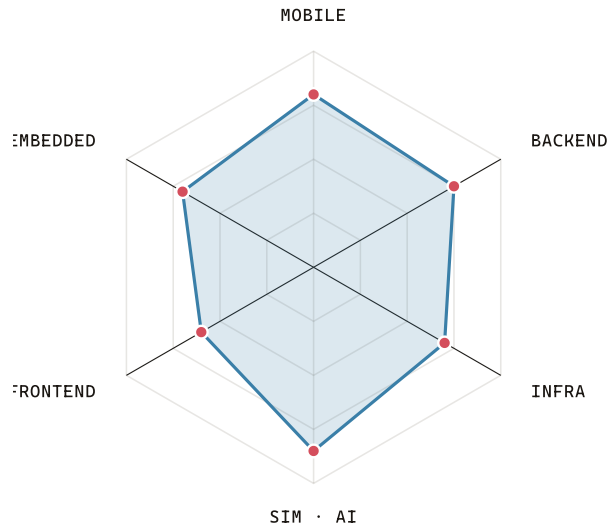
프로젝트에서 실제로 손에 익은 도구들.

STRENGTH MAP

어떤 영역에 무게가 실렸나

네 개의 프로젝트에서 실제로 손에 익은 정도를 분포로 본 것. 절대 점수가 아니라 어디에 시간을 쏟았나의 그림에 가깝다.

- **MOBILE** — Flutter · IMO 앱 아키텍처/계층 통합
- **BACKEND** — Spring Boot · Django DRF · API 명세 정합
- **INFRA** — Docker · Jenkins · Caddy · Nginx · EC2
- **SIM · AI** — Isaac Lab · RL navigation · 멀티 도킹
- **FRONTEND** — Vue.js · WebRTC 시그널링 · UI 통합
- **EMBEDDED** — Raspberry Pi · 앱-디바이스 핸드셰이크



Languages

프로젝트에서 주력으로 쓴 언어



Python



Dart



Java



TypeScript



JavaScript

Frontend & Mobile

앱·웹 클라이언트



Flutter



Vue.js



React

Backend

API·서비스 서버



Spring Boot



Django



Django REST Framework



Node.js

Realtime · Data

실시간 통신과 데이터 저장



WebSocket



WebRTC



Redis



MySQL



PostgreSQL

Infra & DevOps

배포·운영 인프라



Docker



Jenkins



Nginx



Caddy

Sim · AI · Embedded

특화 기술 — 시뮬레이션부터 디바이스까지



Isaac Lab



Reinforcement Learning



ROS



Raspberry Pi

Tools

협업·이슈·디자인



Git



GitLab



Figma



Jira



Mattermost



Notion

Stories

가장 무게 있게 작업한 네 편의 이야기.

STORY NO. 01

2026.04 - 2026.05 · 6주 · 자율PJT

IMO

스마트 글래스로 운동 자세를 보고, 라즈베리파이에서 추정된 데이터를 앱이 받아 통계·코치로 돌려주는 풀스택 운동 코치. 디바이스-서버-앱 3개 계층을 사용자 의도에 맞게 묶는 게 과제였다.

46건

MERGED MR

3계층

E2E 통합

 Flutter  Spring Boot  Django  Redis  Docker  Raspberry Pi +2

read the story →

STORY NO. 02

2026.02 - 2026.04 · 6주 · 특화PJT

AMR 시뮬레이션 플랫폼

Isaac Lab 위에서 자율이동로봇(AMR) 다대수가 동시에 학습·도킹·서비스 사이클을 도는 시뮬레이션 환경. 단일 로봇 학습을 6/12/18대 스케일링까지 끌어올리는 게 과제였다.

50건

MERGED MR

6 → 18대

스케일링 폭

 Isaac Lab  RL  Python  ROS  Docker  Jenkins +2

read the story →

STORY NO. 03

2026.01 - 2026.02 · 6주 · 공통PJT

itda

WebRTC로 다인 협업을 묶고 통합 대시보드 UI에서 룸·타임라인·세션을 한 번에 관리하는 AI 영상 제작 협업 플랫폼.

3채널

실시간 협업 흐름

6명

WEBRTC MESH 기준

✓ Vue.js

🔗 WebRTC

🔌 Spring Boot

📦 Node.js

🚢 Docker

📦 S3

read the story →

STORY NO. 04

2025.12 · 1주 · 공통PJT

MovieApp

영화 추천·한줄평·마이페이지를 갖춘 영화 커뮤니티. 첫 풀스택 협업 프로젝트로, 사용자 콘텐츠 흐름(좋아요·한줄평·나중에 볼)을 끝에서 끝까지 묶는 게 과제였다.

5건

MERGED MR

4흐름

직접 말은 기능

✓ Vue.js

🔴 Django REST Framework

📦 TMDB API

read the story →

Notes

2025.12 ~ 2026.05

SSAFY 14기

교육생 · 4개 프로젝트 완수

- 관통PJT — 영화 커뮤니티 (3인 팀, 마이페이지·한줄평·추천)
- 공통PJT itda — 화상 협업 학습 플랫폼 (6인 팀, WebRTC·UI 통합)
- 특화PJT AMR Sim — 다중 AMR 자율주행 시뮬레이션 (5인 팀, Isaac Lab/RL·인프라)
- 자율PJT IMO — 스마트 글래스 운동 코치 (5인 팀, Flutter·백엔드·라즈베리파이 통합)

Education & Certs

EDUCATION SSAFY 14기 (Samsung SW Academy For Youth)
2025.07 – 2026.05 수료 예정

수료 예정

Contact

새로운 팀, 새로운 문제,
새로운 계층의 통합 —

어디든 끝에서 끝까지 책임지고 싶습니다.

Email →

GitLab ↗ (<https://lab.ssafy.com/ljin2091>)

IMO

스마트 글래스로 운동 자세를 보고, 라즈베리파이에서 추정한 데이터를 앱이 받아 통계·코치로 돌려주는 풀스택 운동 코치. 디바이스-서버-앱 3개 계층을 사용자 의도에 맞게 묶는 게 과제였다.

Flutter

Spring Boot

Django

Redis

Docker

Raspberry Pi

WebSocket

Gemini API

01 BACKGROUND

PROBLEM

운동 코칭 앱은 많지만, 실제로 자세를 "본다"는 부분은 거의 다 사용자가 직접 영상을 찍어 올려야 했다. 그러다 보니 한 세트 끝날 때마다 동작이 끊겨 운동에 몰입하기 어렵고, 코칭 데이터도 그때그때만 남았다. 우리는 사용자가 운동 중 시선을 떼지 않고도 자세 피드백·통계·코치를 받을 수 있어야 한다고 봤다.

OUR APPROACH

스마트 글래스(촬영) → 라즈베리파이(자세 추정) → 백엔드(세션·통계·챗봇) → Flutter 앱(피드백·기록) 4계층을 한 흐름으로 묶어, 사용자는 글래스만 쓰고 있으면 나머지는 자동으로 굴러가는 운동 사이클을 만들자.

EVIDENCE · NOTES

- [채우기] 시장 조사 데이터: 예) 홈트레이닝 사용자 N% 가 자세 피드백을 받지 못해 부상 경험
- [채우기] 사용자 인터뷰: 예) "운동 중 핸드폰을 만지면 흐름이 끊긴다"

02 SNAPSHOT

MERGED MR

46건

Flutter · backend · device 작업 반영

E2E 통합

3계층

Flutter ↔ backend ↔ Raspberry Pi

03 ROLE

TEAM

5인 팀 (앱·백·디바이스 분담)

MY SCOPE

Flutter 앱 초기 아키텍처, 백엔드 세션/통계 API, 라즈베리파이 ↔ 앱 실시간 핸드셰이크 프로토콜, Redis/Docker 환경 통합

04 VISUALS

대표 흐름 보드

PREVIEW

실제 앱 캡처가 남아 있지 않아, 포트폴리오용으로 핵심 사용자 흐름을 다시 정리한 보드.

 IMO 앱의 온보딩, 운동 시작, 통계/챗봇 흐름을 요약한 다이어그램

캐릭터 자산

SCREEN

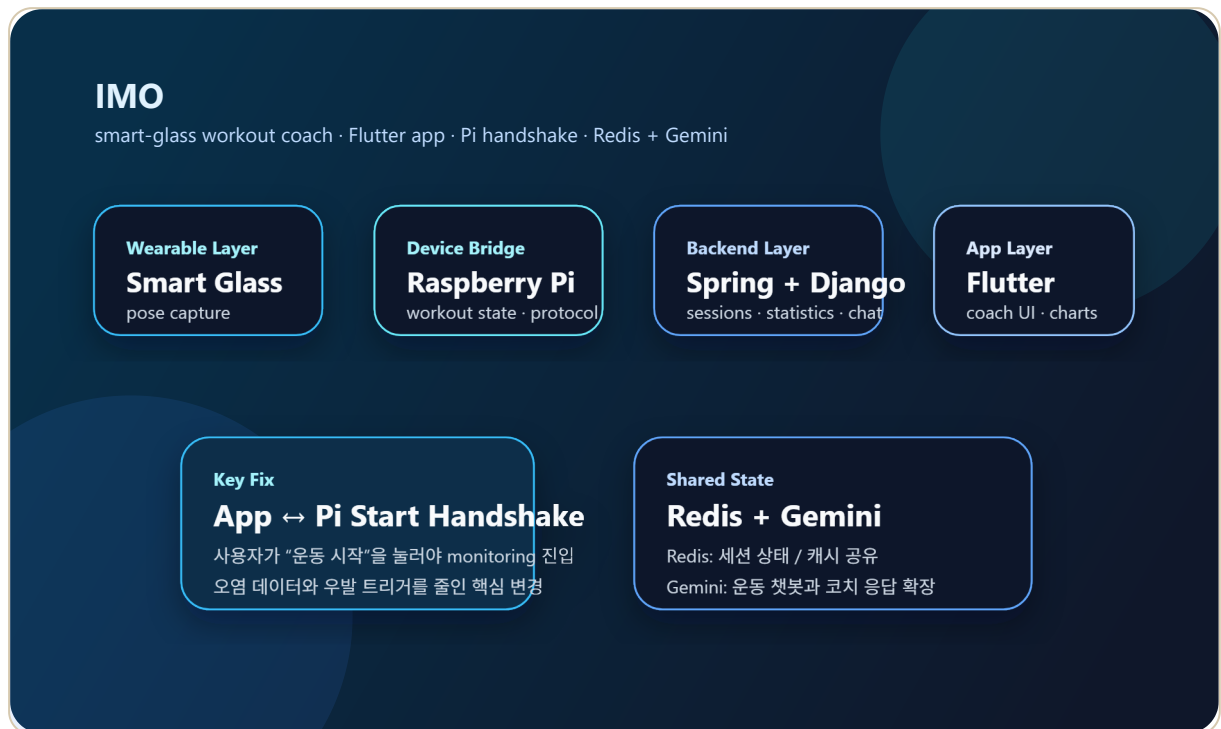
운동 종료 후 결과 대기 흐름에 활용한 대표 캐릭터 자산. 앱의 친화적 톤을 만드는 요소였다.



시스템 아키텍처

스마트 글래스, Pi, 서버, 앱, Redis/Gemini가 어떻게 연결되는지 포트폴리오용으로 재구성한 아키텍처.

ARCH



05 PROBLEM SOLVING

문제를 어떻게 정의했고, 어떤 구조적 판단으로 풀었는지.

01

디바이스가 사용자 의도를 무시하고 자동 진입했다



PROBLEM

캘리브레이션이 성공하면 라즈베리파이가 곧바로 monitoring 상태로 진입해 운동 데이터를 쌓기 시작했다. 사용자가 글래스를 아직 안 쓴 상태에서도 세션이 시작돼 누적 데이터가 오염됐다.



APPROACH

Pi 단독 결정 → 앱-Pi 양방향 핸드셰이크로 전환.
PiMessageType.startWorko 메시지를 신설해 앱에서 "운동 시작" 버튼을 눌러야만 Pi가 monitoring으로 들어가도록 흐름을 바꿨다.
WorkoutRepository 계층까지 변경 전파.



RESULT

세션이 사용자 동의 시점에만 시작되도록 바뀌어 오염 데이터가 사라졌고, 클래스 착용 전 우발 트리거를 0건으로 줄였다.

↗ MR !83 · 앱-Pi 양방향 운동 시작 핸드셰이크 (https://lab.ssafy.com/s14-final/S14P31C203/-/merge_requests/83)

02

주간 추세 막대가 잘못된 요일에 찍혔다



PROBLEM

월요일에 운동을 3번 하면 막대가 일·월·화 자리에 1개씩 분산되어 표시됐다. 백엔드가 세션별 trend values를 순서대로 append하는 구조였고, 앱은 borderRadius=999인 막대를 그대로 쌓아 모양도 깨졌다.



APPROACH

백엔드는 일자별 7일 슬롯에 매핑하도록 집계 로직을 재설계했다. 앱은 막대 모양을 슬롯 단위로 정상화하고 같은 날 다중 세션을 한 막대로 합쳐 그리도록 수정했다.



RESULT

막대 위치와 모양이 모두 정상화되어 사용자가 주간 운동 패턴을 한눈에 읽을 수 있게 됐다.

↗ MR !101 · 주간 추세 일자별 7일 슬롯 매핑 (https://lab.ssafy.com/s14-final/S14P31C203/-/merge_requests/101)

! PROBLEM

달력 화면은 한 달치 데이터와 마커가 필요하고, 리스트 화면은 날짜/타입 필터와 페이지징이 필요한데 기존 GET /sessions는 그걸 한 호출로 주지 못해 화면이 2~3번 호출해야 했다.

→ APPROACH

쿼리 파라미터를 spec(API-07)에 맞춰 date / month / exerciseType + page/size로 정리했다. 응답은 sessions 배열 + exerciseDates(달력 마커) + pagination을 한 번에 담은 구조로 통일했다.

✓ RESULT

달력 화면은 1회 호출로 한 달치를 받고, 리스트 화면도 동일 엔드포인트로 처리해 클라이언트 코드 분기를 줄였다.

↗ MR !25 · GET /sessions 명세 정합 (https://lab.ssafy.com/s14-final/S14P31C203/-/merge_requests/25)

06 TECH DECISIONS

Flutter

Android/iOS 동시 출시가 필요했고 디바이스 통신이 핵심이라 빠른 빌드-실행 검증 사이클이 중요했다. React Native 대비 단일 코드베이스 안정성을 우선해 선택했다.

Redis

Pi-앱-서버 3자가 운동 세션 상태를 공유하는 흐름에서 DB polling보다 in-memory 키로 상태를 주고받는 편이 단순했다. 운영에선 외부 미노출로 격리해 배포 리스크를 낮췄다.

Docker Compose 환경 분리

로컬은 Redis 노출, EC2는 미노출처럼 같은 서비스를 환경별 compose 파일로 분기해 배포 실수를 줄였다.

07 OUTCOMES

세션 목록 데이터 페치

1회 호출

달력 + 리스트 화면을 단일
엔드포인트로 통합

주간 추세 시각화 기준

7일 슬롯

세션 append 방식에서 일
자 매핑 방식으로 수정

운동 시작 우발 트리거

0건

사용자 버튼 기반 핸드셰이
크 도입 후

08 SELECTED MRS

MR !83 · 앱-Pi 양방향 운동 시작 핸드셰이크 (https://lab.ssafy.com/s14-final/S14P31C203/-/merge_requests/83)

사용자 버튼을 기준으로 monitoring 진입 시점을 제어한 핵심 변경.

MR !25 · GET /sessions 명세 정합 (API-07) (https://lab.ssafy.com/s14-final/S14P31C203/-/merge_requests/25)

달력과 리스트 화면을 한 엔드포인트로 맞추는 API 정리.

MR !101 · 주간 추세 7일 슬롯 매핑 (https://lab.ssafy.com/s14-final/S14P31C203/-/merge_requests/101)

통계 집계와 앱 시각화가 어긋나던 문제를 함께 수정.

09 RETROSPECTIVE

LESSON

디바이스-서버-앱 세 계층이 얹힌 흐름은
"누가 상태의 주인인가"를 먼저 못 박지 않음

NEXT TIME

다음에는 첫 주에 상태 머신 다이어그램부터
합의하고, 각 메시지가 어떤 권한을 위임하는
지 코드 주석과 타입 수준에서 명시하겠다.

면 작은 트리거 하나가 전체를 오염시킨다는
걸 핸드셰이크 이슈에서 배웠다.

[다른 프로젝트 보기](#)

[GitLab repo ↗](#) (<https://lab.ssafy.com/s14-final/S14P31C203>)

AMR 시뮬레이션 플랫폼

Isaac Lab 위에서 자율이동로봇(AMR) 다대수가 동시에 학습·도킹·서비스 사이클을 도는 시뮬레이션 환경. 단일 로봇 학습을 6/12/18대 스케일링까지 끌어올리는 게 과제였다.

 Isaac Lab  RL  Python  ROS  Docker  Jenkins  EC2  Caddy

01 BACKGROUND

PROBLEM

[채우기 — 한두 문단] 물류·서비스 현장에서 AMR을 다대수로 운용하려면 어떤 문제가 있는지, 왜 시뮬레이션부터 시작해야 했는지. 예: 실기 테스트 비용, 다대수 동시 학습이 어려운 점, 도킹·서비스 사이클 검증 필요.

OUR APPROACH

[채우기] Isaac Lab 위에 6/12/18대 스케일링 환경 + 도킹 이후 서비스 사이클을 자동화하는 학습 환경을 만들어 실기 부담 없이 검증한다.

EVIDENCE · NOTES

- [채우기] 산업/시장 데이터 (예: AMR 시장 규모, 다대수 운용 실패율)
- [채우기] 기술적 관찰 (예: 단일 로봇 학습은 잘 되는데 다대수는?)

02 SNAPSHOT

MERGED MR

50건

실험 환경 · 배포 인프라 · 런타임 반영

스케일링 폭

6 → 18대

동일 코드베이스 기준

03 ROLE

TEAM

5인 팀 (시뮬·학습·인프라
분담)

MY SCOPE

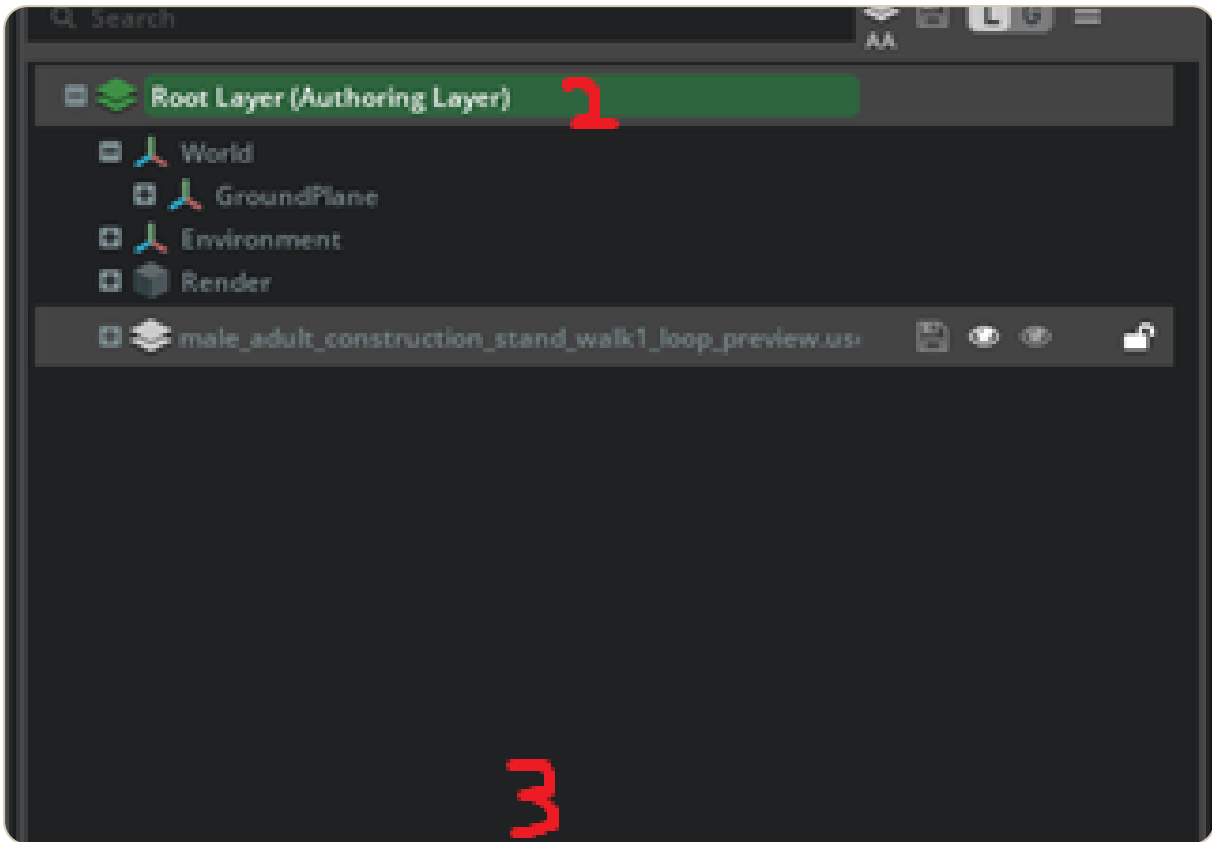
AMR navigation 학습 환경
(asset/config/scene/planner) 구성, 멀티 도킹 서비스 런
타임, 스케일링 실험 환경, EC2/Caddy/Jenkins 배포 인프라

04 VISUALS

시뮬레이터 맵 캡처

SCREEN

실험 맵에 오버레이 자산을 엮어 시나리오를 구성할 때 사용한 실제 시뮬레이터 화면.



실험/배포 아키텍처

ARCH

학습 환경, 런타임, 텔레메트리, 배포 경로를 한 장으로 정리한 포트폴리오용 아키텍처.

AMR 시뮬레이션 플랫폼

Isaac Lab · multi-docking runtime · 6/12/18대 scaling · EC2/Caddy/Jenkins

Simulation Core

Isaac Lab

RL env · assets · scenes · planner

Runtime Loop

play_navigation_multi.p

dock → retreat → returning → done

Observability

Dashboard · Telemetry

status mapping · KPI snapshot

Training Variants

6 / 12 / 18

same codebase · config-only scale-up

Delivery Path

GitLab → Jenkins → Caddy → EC2

build, deploy, verify on the same GPU-friendly environment

05 PROBLEM SOLVING

문제를 어떻게 정의했고, 어떤 구조적 판단으로 풀었는지.

01

멀티 AMR 도킹 후 서비스 흐름이 끊겼다

! PROBLEM

도킹까지는 학습이 잘 되는데, 도킹 이후의 retreat / returning / done 사이클이 정의되지 않아 다음 작업으로 넘어가지 못했다.

→ APPROACH

play_navigation_multi.py를 추가하고 도킹 이후 retreat → returning → done 서비스 사이클을 큐 기반으로 정의했다.
service-v1 학습 환경을 함께 추가해 런타임과 학습 기준을 맞췄다.

✓ RESULT

도킹 이후에도 서비스 런타임이 끊기지 않는 3단계 후속 흐름을 만들었고, 멀티 도킹 시나리오를 끝까지 반복 실행할 기반을 확보했다.

↗ MR !110 · 멀티 도킹 서비스 런타임 및 service-v1 학습 환경 (https://lab.ssafy.com/s14-mobility-smarthome-sub1/S14P21C206/-/merge_requests/110)

02

학습 환경을 늘릴 때마다 환경/경로 설정을 다시 뜯어야 했다

! PROBLEM

단일 AMR navigation 코드를 멀티대로 늘리면 asset/config 경로가 깨지고 scene이 충돌해, 실험 대수를 바꿀 때마다 코드를 복붙해야 했다.

→ APPROACH

isaacsim 경로를 rl_navigation 패키지로 재구성하고 실행 스크립트와 자산 참조를 새 레이아웃에 맞췄다. 패키지 단위로 분리해 N대 환경 추가가 config 변경만으로 가능하도록 만들었다.

✓ RESULT

6/12/18대 스케일링 실험을 동일 코드베이스에서 분기 없이 실행할 수 있게 됐다.

↗ MR !156 · AMR 최적대수 스케일링 실험 환경 (https://lab.ssafy.com/s14-mobility-smarthome-sub1/S14P21C206/-/merge_requests/156)

06 TECH DECISIONS

Isaac Lab

GPU 병렬 시뮬레이션과 RL 학습 틀체인이 한곳에 있어, 학습 환경과 플레이 테스트를 같은 문맥에서 돌리기 쉬웠다. Gazebo/Webots보다 다대수 실험 파이프라인 일체화에 유리했다.

Caddy reverse proxy

nginx 대비 HTTPS 자동화가 단순하고, EC2에서 서비스 주소를 빠르게 정리할 수 있어 배포 셋업 시간을 줄였다.

Jenkins on EC2

Isaac Sim/학습 스택은 무겁고 GPU·OS 의존이 커서 GitHub Actions보다 셀프호스팅이 재현성 면에서 유리했다. Jenkins로 build → deploy → verify를 같은 서버 맥락에서 돌리게 했다.

07 OUTCOMES

AMR 스케일링 실험

6 → 18대

같은 코드베이스로 대수만 바꿔 검증

도킹 이후 서비스 사이클

3단계

retreat → returning → done

프로젝트 MERGED MR

50건

실험·인프라·연동 작업 누적 반영

08 SELECTED MRS

MR !110 · 멀티 도킹 서비스 런타임 및 service-v1 학습 환경 (https://lab.ssafy.com/s14-mobility-smarthome-sub1/S14P21C206/-/merge_requests/110)

도킹 이후 서비스 사이클과 새 학습 환경을 함께 추가한 대표 MR.

MR !156 · 6/12/18대 스케일링 실험 환경 (https://lab.ssafy.com/s14-mobility-smarthome-sub1/S14P21C206/-/merge_requests/156)

같은 코드베이스에서 다대수 실험을 돌릴 수 있게 한 확장 작업.

MR !21 · Docker and EC2 deployment setup (https://lab.ssafy.com/s14-mobility-smarthome-sub1/S14P21C206/-/merge_requests/21)

로컬 검증과 EC2 배포 경로를 연결한 인프라 시작점.

09 RETROSPECTIVE

LESSON

다대수 RL은 정책 튜닝보다도 경로, 자산 참조, 미션 파일 구조가 먼저 흔들리기 쉬웠다. 학습 코드와 플레이 런타임의 경계를 초반에 잡는 일이 생각보다 더 중요했다.

NEXT TIME

다음에는 첫 주부터 env/config/log 포맷을 고정하고, 1대 → 6대 → 12대 smoke 시나리오를 자동화해 스케일 문제를 학습 전에 먼저 드러내겠다.

다른 프로젝트 보기 [GitLab repo ↗ \(https://lab.ssafy.com/s14-mobility-smarthome-sub1/S14P21C206\)](https://lab.ssafy.com/s14-mobility-smarthome-sub1/S14P21C206)

← Home

2026.01 - 2026.02 · 6주 · 공통PJT

itda

WebRTC로 다인 협업을 묶고 통합 대시보드 UI에서 룸·타임라인·세션을 한 번에 관리하는 AI 영상 제작 협업 플랫폼.

Vue.js

WebRTC

Spring Boot

Node.js

Docker

S3

01 BACKGROUND

PROBLEM

[채우기 — 한두 문단] AI 영상 제작 협업 시장의 문제 — 시나리오/썸/편집이 별개 도구에 흩어져 있어 협업이 어려움. 화상 협업이 통합돼 있지 않으면 흐름이 끊긴다.

OUR APPROACH

[채우기] 시나리오·썸 편집·실시간 화상(WebRTC Mesh)·타임라인을 한 화면에서 처리할 수 있게 묶는다.

EVIDENCE · NOTES

- [채우기] 예: 영상 제작 협업 도구가 분산돼 있어 평균 N개 도구를 동시에 사용
- [채우기] 예: 화상 통화는 Zoom·Meet 등 별도 — 작업 흐름 단절

02 SNAPSHOT

실시간 협업 흐름

3채널

chat · presence · rtc

WEBRTC MESH 기준

6명

문서와 UI 스코프를 동일 기준으로 맞춤

03 ROLE

TEAM

6인 팀

MY SCOPE

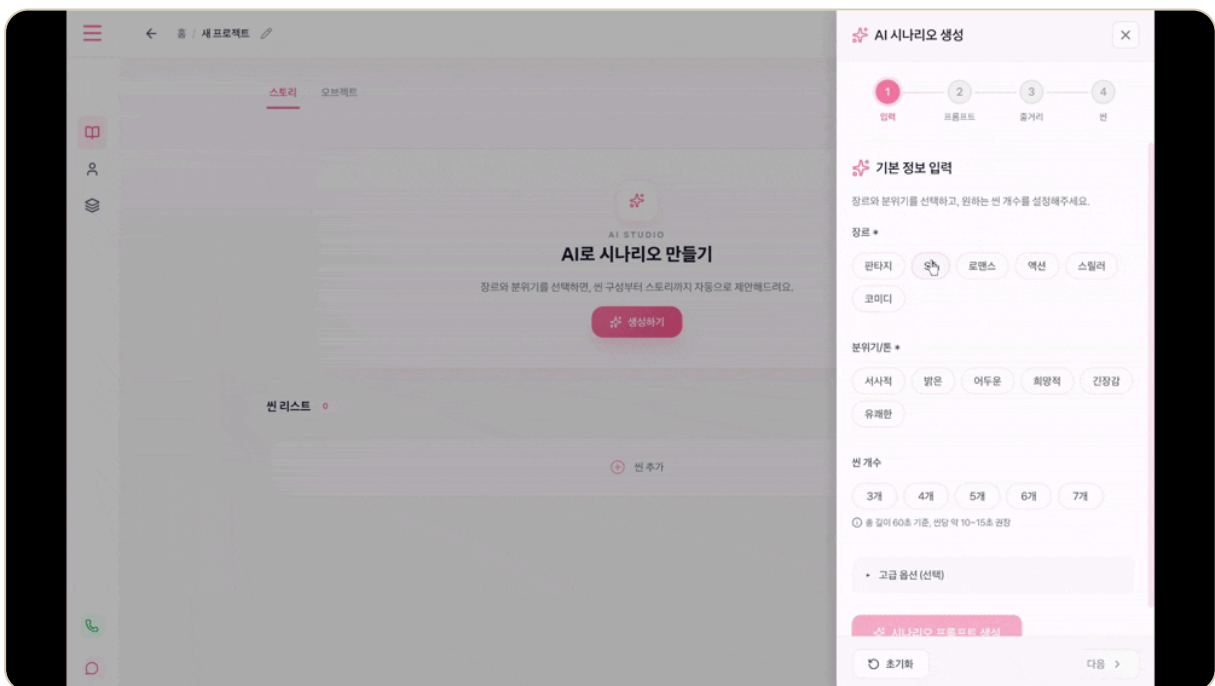
WebRTC 시그널링 처리, 사이드바·MiniTimeline UI 통합,
라우팅 정리

04 VISUALS

AI 시나리오 생성

장르, 분위기, 씬 개수를 입력해 시나리오를 생성하는 실제 동작 GIF.

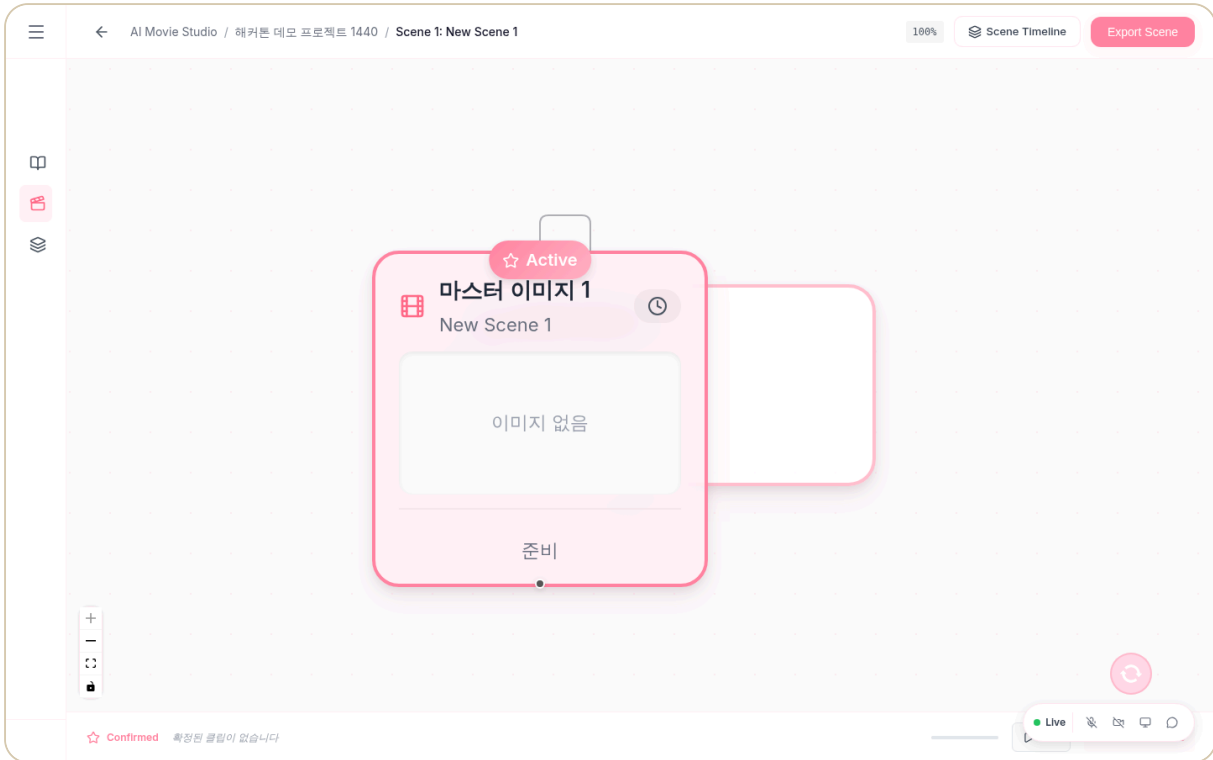
GIF



씬 에디터

MiniTimeline과 협업 UI가 들어가는 핵심 작업 공간. 노드 기반 편집의 중심 화면이다.

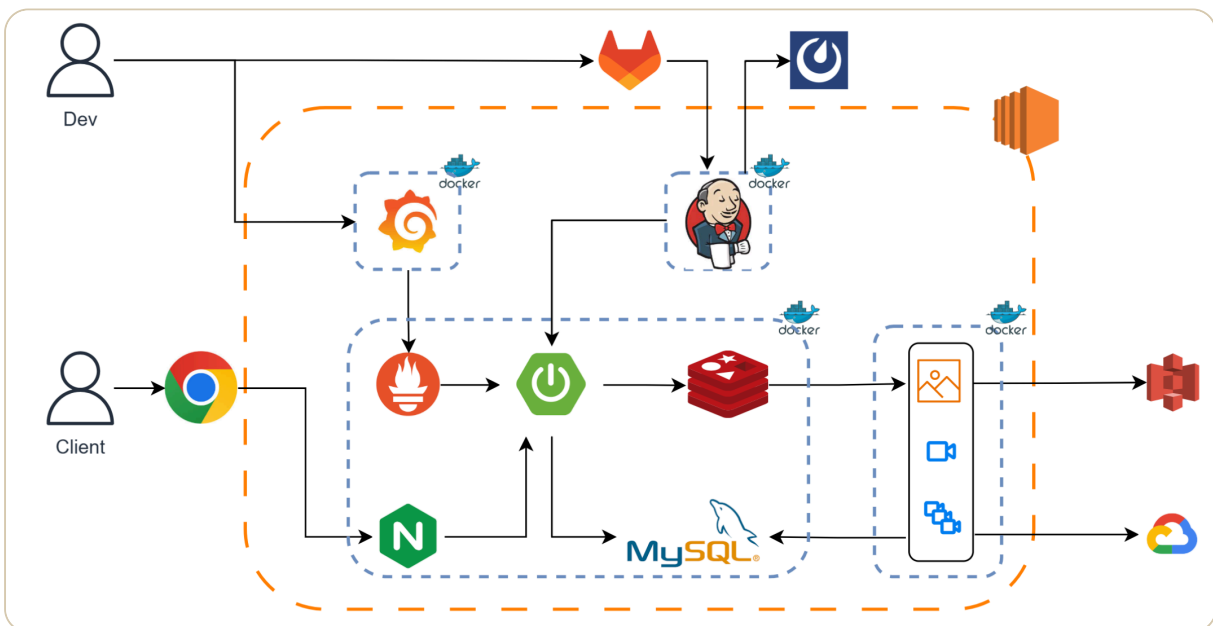
SCREEN



시스템 아키텍처

프로젝트 README에 포함돼 있던 실제 아키텍처 이미지. 협업, AI 작업, 배포 흐름이 한 장에 담겨 있다.

ARCH



05 PROBLEM SOLVING

문제를 어떻게 정의했고, 어떤 구조적 판단으로 풀었는지.

01

협업 채널이 페이지마다 흩어져 실시간 상태가 어긋났다



PROBLEM

프로젝트 상세, 씬 편집, 타
임라인 레이아웃이 각각 소
켓·프레즌스·RTC 연결을 따
로 폼으면 페이지 이동 때 구
독 해제와 재연결 순서가 어
긋나 협업 바가 쉽게 불안정
해졌다.



APPROACH

collabStore를 중심으로
chat / presence / rtc 3
개 채널을 projectId 기준
으로 묶고,
socketManager와
peerConnectionService
의 join → subscribe →
media 초기화 순서를 고정
했다. 레이아웃은 공통 사이
드바와 협업 패널 슬롯을 공
유하도록 정리했다.



RESULT

대시보드·상세·에디터에서
같은 협업 상태를 재사용할
수 있는 구조를 만들었고,
최대 6명 Mesh 전제를 코
드와 문서 양쪽에 맞춰 정렬
했다.

↗ 협업 채널 3종(chat/presence/rtc) + 6명 Mesh 설계

02

레이아웃과 라우팅 충돌 때문에 UI 통합 속도가 느렸다



PROBLEM

MiniTimeline, 사이드바, 프로젝트/썸 편집 레이아웃을 여러 브랜치가 동시에 건드리면서 클릭 흐름과 URL 파라미터가 엇갈릴 때 화면 전환이 쉽게 깨졌다.



APPROACH

공통 레이아웃과 라우터 경계를 다시 나누고, projectId/scenelId 파라미터를 기준으로 페이지 책임을 정리했다. 충돌이 잦았던 공통 셀 영역은 merge 단위를 잘게 쪼개 반영했다.



RESULT

프로젝트 상세와 썸 편집 진입 흐름을 다시 안정화했고, 협업 UI와 MiniTimeline을 같은 화면 계층에 올릴 수 있게 됐다.



MR !70 · 라우팅 수정 (https://lab.ssafy.com/s14-webmobile1-sub1/S14P11C205/-/merge_requests/70)

06

TECH DECISIONS

WebRTC (P2P)

문서상 최대 6명 Mesh를 협업 통화 범위로 잡고 있었고, 음성·화면공유 중심의 소규모 방을 빠르게 MVP로 붙이는 게 우선이었다. SFU보다 구현 복잡도와 운영 비용이 낮아 P2P를 택했다.

Vue.js

Composition API와 Pinia로 편집기 상태와 협업 상태를 나누기 쉬웠고, Vue Flow를 포함한 팀의 기존 프론트 자산과도 궁합이 좋아 빠른 UI 통합에 유리했다.

STOMP Presence 채널

chat-presence-rtc를 브로드캐스트와 1:1 큐 라우팅으로 분리하면 레이아웃이 달라도 협업 UI를 같은 규칙으로 재사용할 수 있었다.

07

OUTCOMES

실시간 협업 구조

MESH 통화 설계 범위

직접 머지한 MR

3채널

chat · presence · rtc를
projectId 기준으로 정리

6명

문서/코드/UI 스코프를 동일
기준으로 통일

2건

데모 안정화와 라우팅 정리
중심

08 SELECTED MRS

MR !70 · 라우팅 수정 (https://lab.ssafy.com/s14-webmobile1-sub1/S14P11C205/-/merge_requests/70)

프로젝트/싹 편집 전환이 흔들리던 경로를 다시 맞춘 정리 MR.

MR !69 · 원상복구 (https://lab.ssafy.com/s14-webmobile1-sub1/S14P11C205/-/merge_requests/69)

데모 직전 흐름 안정화와 UI 회귀를 되돌린 보수 MR.

09 RETROSPECTIVE

LESSON

실시간 협업은 미디어 처리보다도 이벤트 라우팅과 재접속 순서를 먼저 고정해야 UI가 안정해진다. 화면은 그 위에 올라오는 결과물에 가깝다.

NEXT TIME

다음에는 collab contract(chat/presence/rtc)와 reconnect 시나리오를 첫 주부터 fixture로 만들고, 레이아웃 공통 셀 변경은 별도 브랜치로 격리하겠다.

[다른 프로젝트 보기](#)

[GitLab repo ↗](#) (<https://lab.ssafy.com/s14-webmobile1-sub1/S14P11C205>)

← Home

2025.12 · 1주 · 관통PJT

MovieApp

영화 추천·한줄평·마이페이지를 갖춘 영화 커뮤니티. 첫 풀스택 협업 프로젝트로, 사용자 콘텐츠 흐름(좋아요·한줄평·나중에 볼)을 끝에서 끝까지 묶는 게 과제였다.

Vue.js

Django REST Framework

TMDB API

01 BACKGROUND

PROBLEM

[채우기 — 한두 문단] 1주짜리 첫 풀스택 협업 프로젝트로, 영화 데이터를 활용해 사용자가 자기 취향을 정리·공유할 수 있는 공간이 필요했다. 단순 카탈로그가 아닌 "내 활동이 모이는 곳".

OUR APPROACH

[채우기] TMDB로 메타데이터 부담을 줄이고, 사용자 활동(좋아요·한줄평·나중에 볼)을 마이페이지에 묶어 한눈에 보이게 한다.

EVIDENCE · NOTES

— [채우기] 예: 영화 추천 서비스가 많지만 한줄평+좋아요+나중에 볼 흐름이 한곳에 묶여 있는 곳이 적음

02 SNAPSHOT

MERGED MR

5건

1주 프로젝트 기준 기능 단위 반영

직접 맡은 기능

4흐름

마이페이지 · 한줄평 · 나중에 볼 · 추천

03 ROLE

TEAM

3인 팀

MY SCOPE

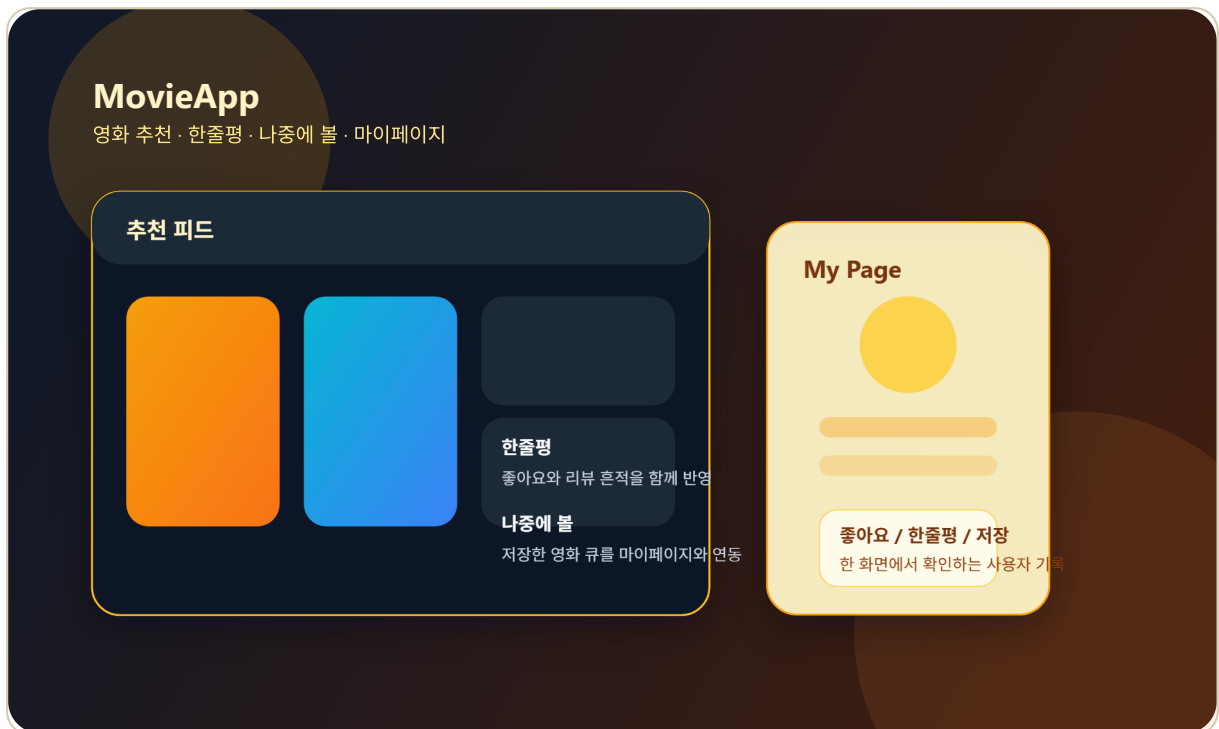
마이페이지(프로필/좋아요/한줄평), 한줄평 작성·조회, 추천
콘텐츠 큐

04 VISUALS

대표 화면 보드

실제 화면 캡처가 남아 있지 않아, 추천 피드와 마이페이지 중심으로 기능 구성을 다시 정리한 포트폴리오용 보드.

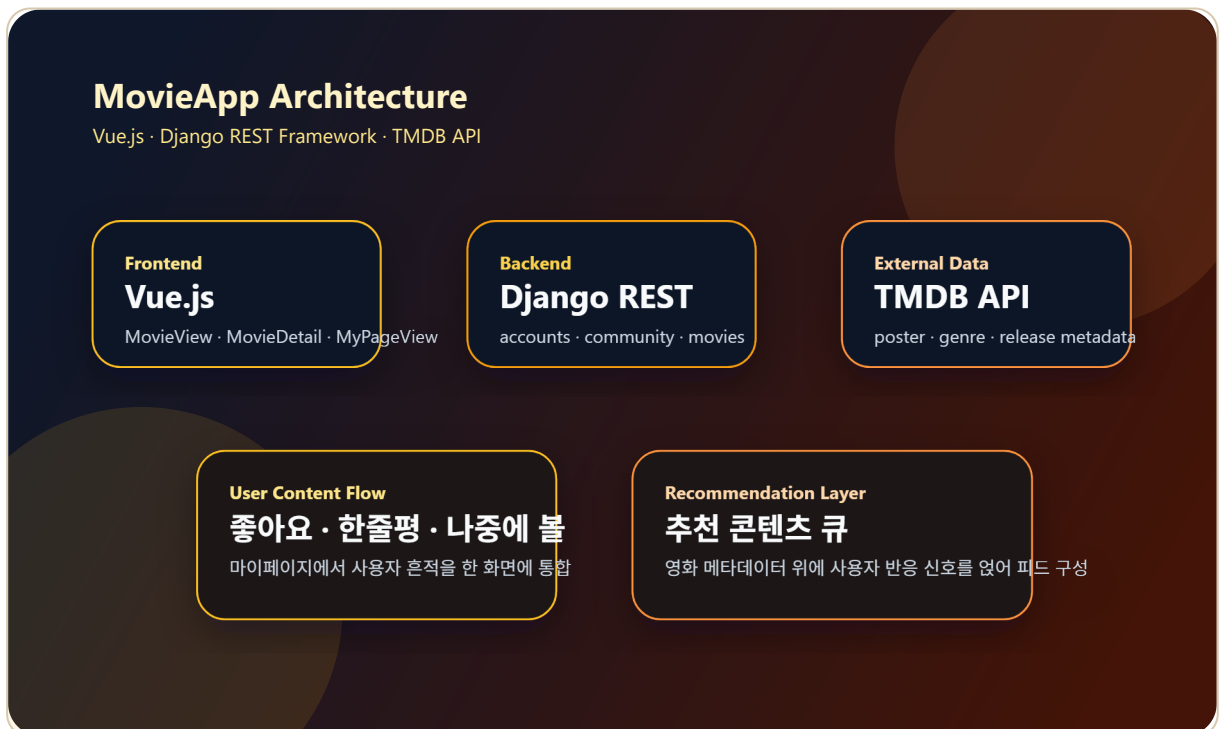
PREVIEW



서비스 구성

Vue 프론트, DRF 백엔드, TMDB 메타데이터 연동, 사용자 콘텐츠 흐름을 설명한 아키텍처.

ARCH



05 PROBLEM SOLVING

문제를 어떻게 정의했고, 어떤 구조적 판단으로 풀었는지.

01 마이페이지가 사용자 활동을 한곳에 모으지 못했다

! PROBLEM

좋아요, 한줄평, 나중에 볼 기능이 영화/커뮤니티/계정 화면에 흩어져 있어 사용자가 자기 활동을 한 눈에 보기 어려웠다.

→ APPROACH

watchlist 관계를 모델과 migration에 추가하고, movies/community/account 뷰를 묶어 MyPageView에서 프로필·저장·리뷰 흐름을 함께 보여주도록 정리했다.

✓ RESULT

마이페이지에서 프로필, 저장 목록, 한줄평 흐름이 연결되며 사용자 중심 화면이 생겼다.

↗ MR !7 · 마이페이지 (https://lab.ssafy.com/ljin2091/final-pjt/-/merge_requests/7)

02 추천 섹션이 영화 메타데이터만 보여주면 밋밋했다

! PROBLEM

단순 최신/인기 목록만 보여주면 사용자 취향이 드러나지 않아 커뮤니티 서비스다운 차별점이 약했다.

→ APPROACH

메인 MovieView와 백엔드 movies/views.py에서 추천 영역을 분리하고, 좋아요·한줄평·나중에 볼 같은 사용자 반응 신호를 추천 큐에 반영하도록 구성했다.

✓ RESULT

메인 피드에 개인화된 추천 블록을 추가해 탐색 흐름을 강화했다.

↗ MR !13 · 좋아할 만한 콘텐츠 (https://lab.ssafy.com/ljin2091/final-pjt/-/merge_requests/13)

06 TECH DECISIONS

Vue.js + DRF

1주 안에 화면과 API를 동시에 붙여야 하는 프로젝트였기 때문에, 프론트와 백을 명확히 나누면서도 빠르게 CRUD를 만들 수 있는 조합이 필요했다. Django ORM과 serializer 덕분에 영화·커뮤니티·계정 도메인을 빨리 엮을 수 있었다.

TMDB API

포스터, 장르, 개봉일 같은 영화 메타데이터를 직접 적재하지 않고, 사용자 기능과 커뮤니티 흐름 구현에 시간을 쓰기 위해 외부 API를 활용했다.

07 OUTCOMES

MERGED MR

5건

기능 단위로 빠르게 쪼개어 반영

사용자 콘텐츠 기능

4흐름

마이페이지 · 한줄평 · 나중
에 볼 · 추천

소규모 협업

3인

첫 풀스택 팀 프로젝트

08 SELECTED MRS

MR !7 · 마이페이지 (https://lab.ssafy.com/ljin2091/final-pjt/-/merge_requests/7)

watchlist 관계와 MyPageView를 포함해 사용자 활동 허브를 만든 대표 MR.

MR !12 · 한줄평 등등 (https://lab.ssafy.com/ljin2091/final-pjt/-/merge_requests/12)

영화 상세와 라우팅을 건드리며 리뷰 작성/조회 흐름을 붙인 작업.

MR !13 · 좋아할 만한 콘텐츠 (https://lab.ssafy.com/ljin2091/final-pjt/-/merge_requests/13)

메인 영화 피드에 추천 블록을 추가한 마무리 기능.

09 RETROSPECTIVE

LESSON

첫 협업에서는 화면보다 데이터 모델을 먼저 맞추는 편이 전체 속도를 더 높인다. 사용자 활동이 여러 화면에 걸치면 관계 설계가 늦을 수록 프론트와 백이 함께 흔들린다.

NEXT TIME

다음에는 관계형 필드와 응답 shape를 먼저 문서화하고, 라우터와 뷰 이름까지 초반에 맞춘 뒤 구현하겠다.

[다른 프로젝트 보기](#)

[GitLab repo ↗](#) (<https://lab.ssafy.com/ljin2091/final-pjt>)